```python
V = 5
edges = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),(3,2,5),(3,1,1),(4,3,-3)]

dist = [999, 999, 999, 999, 999]
dist[0] = 0

for i in range(4):
    for u, v, w in edges:
            if dist[u] + w < dist[v]:
                        dist[v] = dist[u] + w

                        print("Vertex Distance")

                        for i in range(5):
                            print(i, dist[i])
```

```
Vertex Distance
0 0
1 -1
2 999
3 999
4 999
Vertex Distance
0 0
1 -1
2 4
3 999
4 999
Vertex Distance
0 0
1 -1
2 2
```

```python
INF = 999

a = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]

n = 4

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] + a[k][j] < a[i][j]:
                a[i][j] = a[i][k] + a[k][j]

for row in a:
    print(row)
```

input

```
[0, 5, 8, 9]
[999, 0, 3, 4]
[999, 999, 0, 1]
[999, 999, 999, 0]
```

```python
a = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

n = 4

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

for row in a:
    print(row)
```

```
[0, 1, 1, 1]
[0, 0, 1, 1]
[0, 0, 0, 1]
[0, 0, 0, 0]
```

```python
for i in range(n):
    d[i][i] = 0

for u, v, w in edges:
    d[u][v] = w
    d[v][u] = w

for k in range(n):
    for i in range(n):
        for j in range(n):
            if d[i][k] + d[k][j] < d[i][j]:
                d[i][j] = d[i][k] + d[k][j]

answer = 0
minimum = n

for i in range(n):
    count = 0

    for j in range(n):
        if i != j and d[i][j] <= threshold:
            count = count + 1

    if count <= minimum:
        minimum = count
        answer = i

print(answer)
```

```
3
```

```python
times = [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]

n = 4
source = 2

d = [999] * (n + 1)
d[source] = 0

for i in range(n - 1):
    for u, v, w in times:
        if d[u] != 999 and d[u] + w < d[v]:
            d[v] = d[u] + w

answer = max(d[1:])

if answer == 999:
    print(-1)
else:
    print(answer)
```

```
2
```

```python
n = 4

flights = [
    [0, 1, 100],
    [1, 2, 100],
    [2, 3, 100],
    [0, 3, 500]
]

src = 0
dst = 3
K = 2

d = [999] * n
d[src] = 0

for i in range(K + 1):
    temp = d[:]

    for u, v, cost in flights:
        if d[u] != 999 and d[u] + cost < temp[v]:
            temp[v] = d[u] + cost

    d = temp

print(d[dst])
```

```
300
```

```python
V = 4

edges = [
    (0, 1, 1),
    (1, 2, -1),
    (2, 3, -1),
    (3, 0, -1)
]

d = [0] * V

for i in range(V - 1):
    for u, v, w in edges:
        if d[u] + w < d[v]:
            d[v] = d[u] + w

found = False

for u, v, w in edges:
    if d[u] + w < d[v]:
        found = True
        break

if found:
    print("Negative Weight Cycle Exists")
else:
    print("No Negative Weight Cycle")
```

```
Negative Weight Cycle Exists
```

```python
a = [
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 0]
]

u = 0
v = 2
n = 3

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

if a[u][v] == 1:
    print("Path Exists")
else:
    print("Path Does Not Exist")
```

```
Path Exists
```

```python
a = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

source = 0
n = 4

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

count = 0

for j in range(n):
    if j != source and a[source][j] == 1:
        count = count + 1

print(count)
```

```
3
```

```python
INF = 999

a = [
    [0, 2, INF],
    [2, 0, 3],
    [INF, 3, 0]
]

n = 3

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] + a[k][j] < a[i][j]:
                a[i][j] = a[i][k] + a[k][j]

diameter = 0

for i in range(n):
    for j in range(n):
        if a[i][j] != INF and a[i][j] > diameter:
            diameter = a[i][j]

print(diameter)
```

```
5
```

```python
INF = 999

a = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

u = 0
v = 3
n = 4

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] + a[k][j] < a[i][j]:
                a[i][j] = a[i][k] + a[k][j]

print(a[u][v])
```

```
6
```

```python
    [0, 1, 1],
    [0, 0, 1],
    [0, 0, 0]
]

n = 3

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

answer = -1

for j in range(n):
    possible = True

    for i in range(n):
        if i != j and a[i][j] == 0:
            possible = False
            break

    if possible:
        answer = j
        break

print(answer)
```

```
2
```

```python
    [0, 0, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
]

n = 4

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

answer = 0
maximum = -1

for i in range(n):
    count = 0

    for j in range(n):
        if i != j and a[i][j] == 1:
            count = count + 1

    if count > maximum:
        maximum = count
        answer = i

print(answer)
```

```
0
```

```python
INF = 999

a = [
    [0, 2, INF],
    [2, 0, 4],
    [INF, 4, 0]
]

n = 3

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] + a[k][j] < a[i][j]:
                a[i][j] = a[i][k] + a[k][j]

diameter = 0

for i in range(n):
    for j in range(n):
        if a[i][j] != INF and a[i][j] > diameter:
            diameter = a[i][j]

print(diameter)
```

```
6
```

```python
a = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]

n = 3

for k in range(n):
    for i in range(n):
        for j in range(n):
            if a[i][k] == 1 and a[k][j] == 1:
                a[i][j] = 1

strong = True

for i in range(n):
    for j in range(n):
        if i != j and a[i][j] == 0:
            strong = False
            break

if strong:
    print("Strongly Connected")
else:
    print("Not Strongly Connected")
```

```
Strongly Connected
```